

PROGRAMMAZIONE 2

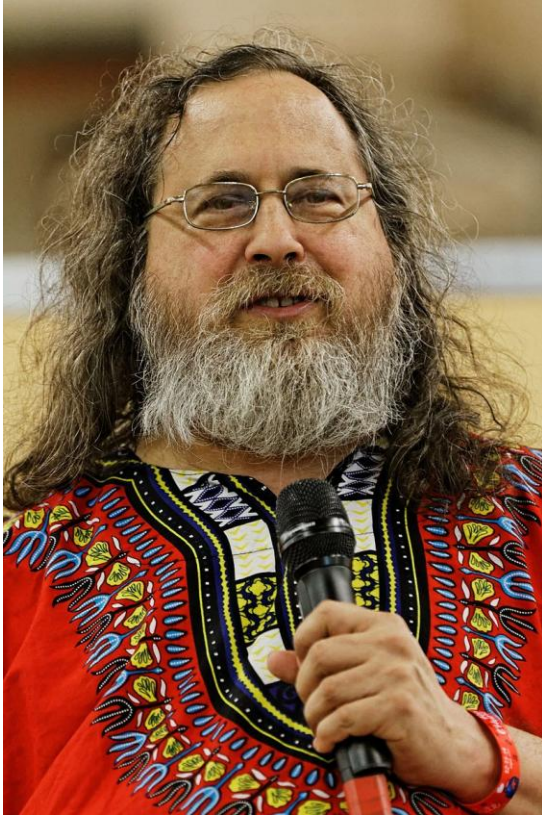
a.a. 2024-2025

GDB (GNU debugger)

INTRODUZIONE A GDB

- GDB è un potente strumento di debugging attualmente mantenuto dalla Free Software Foundation
- Originariamente sviluppato da Richard Stallman nel 1986
- Supporta vari linguaggi di programmazione, tra cui il C
- E' stato concepito per essere usato a riga di comando (shell testuale), anche se numerosi strumenti di sviluppo dotati di interfaccia grafica si possono collegare a esso, offrendo:
 - + Più facilità d'uso
 - Meno funzionalità
- Il modo migliore per conoscerlo è tramite riga di comando. Inoltre in certi casi (shell SSH) si ha a disposizione solo questa modalità d'uso

<https://www.sourceware.org/gdb/>



Richard Stallman
(Free Software Foundation)



Logo of the GNU project

IL MONDO DEI BUG

- I bug si annidano inevitabilmente anche nel codice sviluppato da ottimi programmatori
- Buona parte del tempo di un programmatore è speso, ahimè, nel trovare e risolvere i bug
- I bug possono produrre:
 - Crash
 - Cicli infiniti
 - Risultati sbagliati
 - Altri casi di comportamento anomalo
- Si differenziano anche tra:
 - Deterministici
 - Dipendenti dall'input
 - Random
- ... con impatto sulla loro **riproducibilità**



GDB: COSA FA

- Dalla prima pagina della guida di GDB:

GDB allows you to see what is going on “inside” another program while it executes—or what another program was doing at the moment it crashed.

- E in particolare permette di:
 - Start your program, specifying anything that might affect its behavior.
 - Make your program stop on specified conditions.
 - Examine what has happened, when your program has stopped.
 - Change things in your program, so you can experiment with correcting the effects of one bug and go on to learn about another

GDB: PREGI E DIFETTI

- + Permette di scovare i bug più ostici, che resistono a strategie di debug più semplici (es: le printf)
- + Può fare virtualmente "qualunque cosa", a patto di capire come fare (900 pagine di user manual...)
- + Può debuggare anche programmi già in esecuzione, e attaccarsi a processi remoti
- + Permette di fare un'analisi "post-mortem" di un programma andato in chash
- L'interfaccia command-line può essere ostica
- Gli infiniti comandi e opzioni possono mettere in soggezione, ma non bisogna scoraggiarsi! Un uso "base" richiede di imparare poche cose

GDB: COSA CI FA VEDERE

- Per un dato processo, GDB permette di osservare/manipolare:
 - l'effetto di ciascuna linea di codice del file sorgente
 - variabili di tutti i tipi
 - lo stack delle varie chiamate a funzione in corso
 - posizioni arbitrarie nella memoria
 - i registri della CPU
 - i diversi (eventuali) thread del processo

GDB: COMPILAZIONE CON OPZIONE -G

- Per debuggare un programma efficacemente, occorre includere nel file oggetto informazioni di debugging generate automaticamente in fase di compilazione con *l'opzione -g*
 - > **gcc -g pippo.c -o pippo**
 - > **gdb pippo**
- Inoltre, sebbene gdb possa debuggare anche codice ottimizzato (-O<n>), è meglio non produrre codice ottimizzato in fase di debug

IL "CORE DUMP"

- Se un programma va in crash, tutte le informazioni del processo vengono salvate in un file detto core dump
- GDB permette di ispezionare questo file come se fosse un processo in esecuzione
- Molto utile per analisi "post mortem"

FLUSSO TIPICO DI UNA "SESSIONE DI DEBUG"

- Inizio sessione di debug:
 - `gdb eseguibile`
- Per passare parametri sulla linea di comando:
 - `gdb --args eseguibile arg1 arg2 ...`
- Entro la sessione di debug:
 - `(gdb) comando1`
 - `(output comando1)`
 - `(gdb) comando2`
 - `(output comando2)`
 - `...`
 - `(gdb) help comando`
 - `...`
- Fine della sessione:
 - `(gdb) quit`

GDB: COMANDI PER CONTROLLO ESECUZIONE

- **b** (**break**): imposta un breakpoint ad una linea specifica del codice sorgente oppure ad una funzione, magari sotto condizione
 - (gdb) b 10
 - (gdb) b miafunzione
 - (gdb) b miafunzione if myvar == 0
- **r** (**run**): inizia esecuzione
- **c** (**continue**): continua esecuzione dopo un breakpoint
- **n** (**next**): esegue la prossima istruzione (senza entrare nelle funzioni chiamate)
- **s** (**step**): esegue la prossima istruzione (entrando nelle funzioni chiamate)
- **f** (**finish**): continua esecuzione fino al termine della funzione corrente
- **<CTRL-C>**: interrompe esecuzione del programma (come se si fosse raggiunto un breakpoint) (utile con loop infiniti)

GDB: ALTRI COMANDI UTILI

- `i b` (`info breakpoints`): mostra i breakpoints attuali
- `d` (`delete`): rimuove il breakpoint n-esimo
 - `(gdb) d 2` rimuove il breakpoint 2
- `watch myvar`: ferma l'esecuzione del programma quando il valore di una variabile (o di una espressione) nel contesto corrente cambia
- `watch myaddr`: stop quando una locazione di memoria viene modificata
- `i wat` (`info watchpoints`): mostra i watchpoint attuali
- `display myvar`: stampa il valore di una variabile (espressione) a ogni arresto dell'esecuzione
- `layout next`: mostra il codice sorgente, con varie opzioni

GDB: COMANDI PER OPERAZIONI SU VARIABILI

- **p** (**print**): stampa il contenuto di una variabile o di un indirizzo in memoria

```
(gdb) p myvar
```

```
(gdb) p &myvar
```

```
(gdb) p argv[1]
```

```
(gdb) p A[10]@5
```

Stampa 5 elementi a partire
dalla pos 10 dell'array A

```
(gdb) p *(long *)0x614c20
```

Stampa un long a partire
dall'indirizzo 0x614c20

- **pt** (**ptype**): stampa il tipo di una variabile
- Possiamo anche cambiare il valore di una variabile/locazione di memoria!:

```
(gdb) set x=4
```

```
(gdb) set {int}0x83040 = 4
```

Memorizza un intero a partire
dall'indirizzo 0x83040

GDB: OPERAZIONI SULLO STACK

- `bt` (`backtrace`): visualizza lo stack di tutte le chiamate con i relativi argomenti
- `f` `framenumbers`: seleziona un frame dello stack
- `up`: si sposta nel chiamante del frame corrente
- `down`: si sposta nel chiamato del frame corrente
- `info locals`: mostra le variabili locali del frame corrente
- `info args`: mostra gli argomenti del frame corrente